

Distributed Systems

Section 4. Communication

Pedro García López

pgarcia@etse.urv.es/

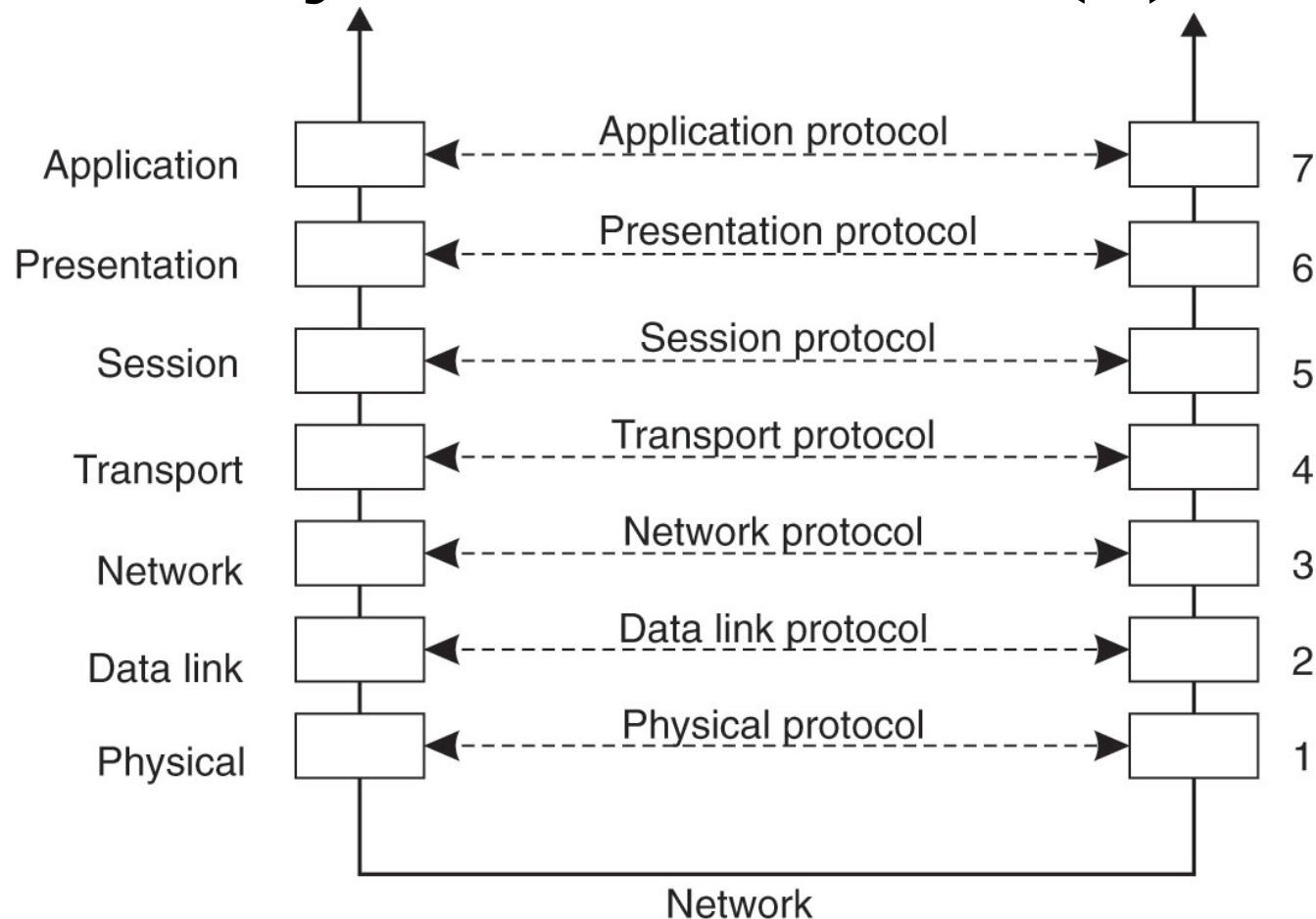
Copyright

- © University Rovira i Virgili
- Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.1 or any later version published by the Free Software Foundation; provided its original author is mentioned and the link to <http://libre.act-europe.fr/> is kept at the bottom of every non-title slide. A copy of the license is available at:
<http://www.fsf.org/licenses/fdl.html>

Communication paradigms

<i>Communicating entities (what is communicating)</i>		<i>Communication paradigms (how they communicate)</i>		
<i>System-oriented entities</i>	<i>Problem- oriented entities</i>	<i>Interprocess communication</i>	<i>Remote invocation</i>	<i>Indirect communication</i>
Nodes	Objects	Message passing	Request- reply	Group communication
Processes	Components	Sockets	RPC	Publish-subscribe
	Web services	Multicast	RMI	Message queues
				Tuple spaces
				DSM

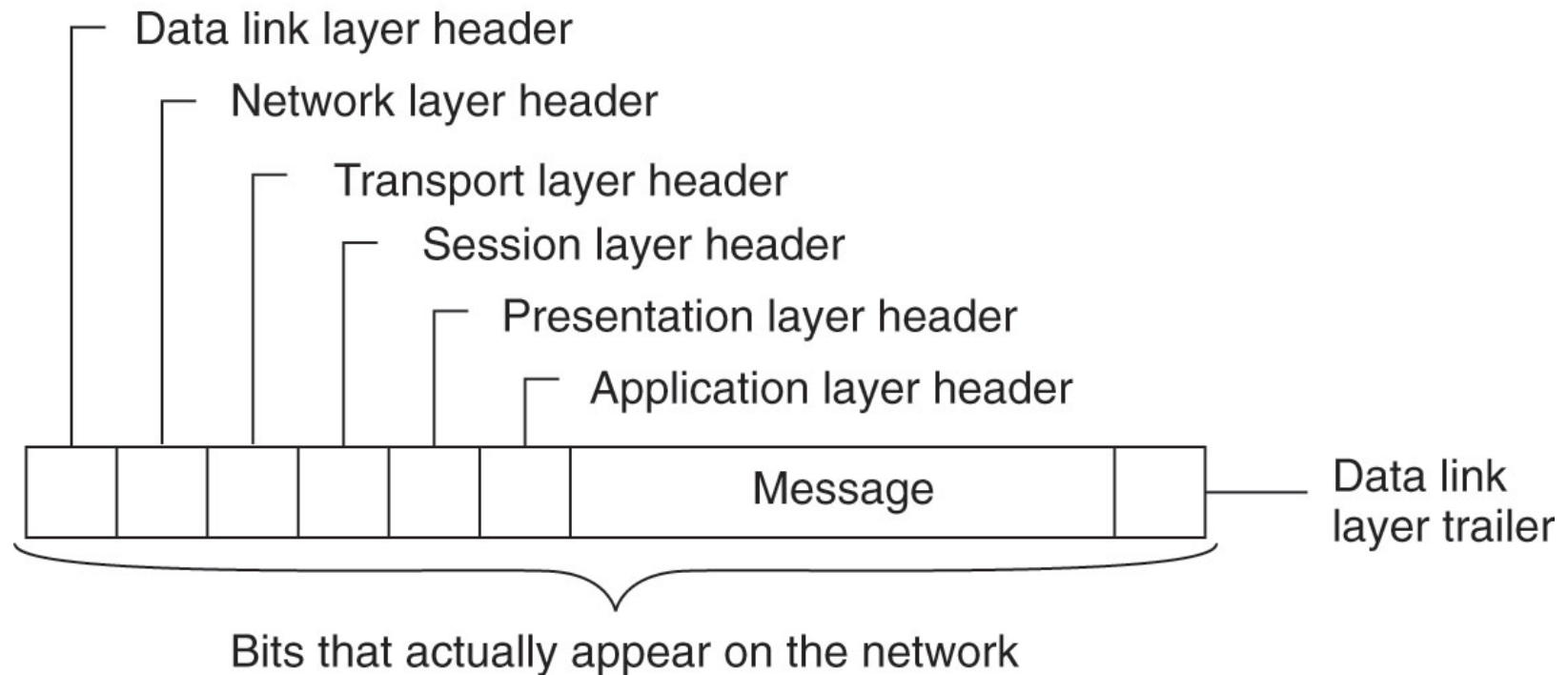
Layered Protocols (1)



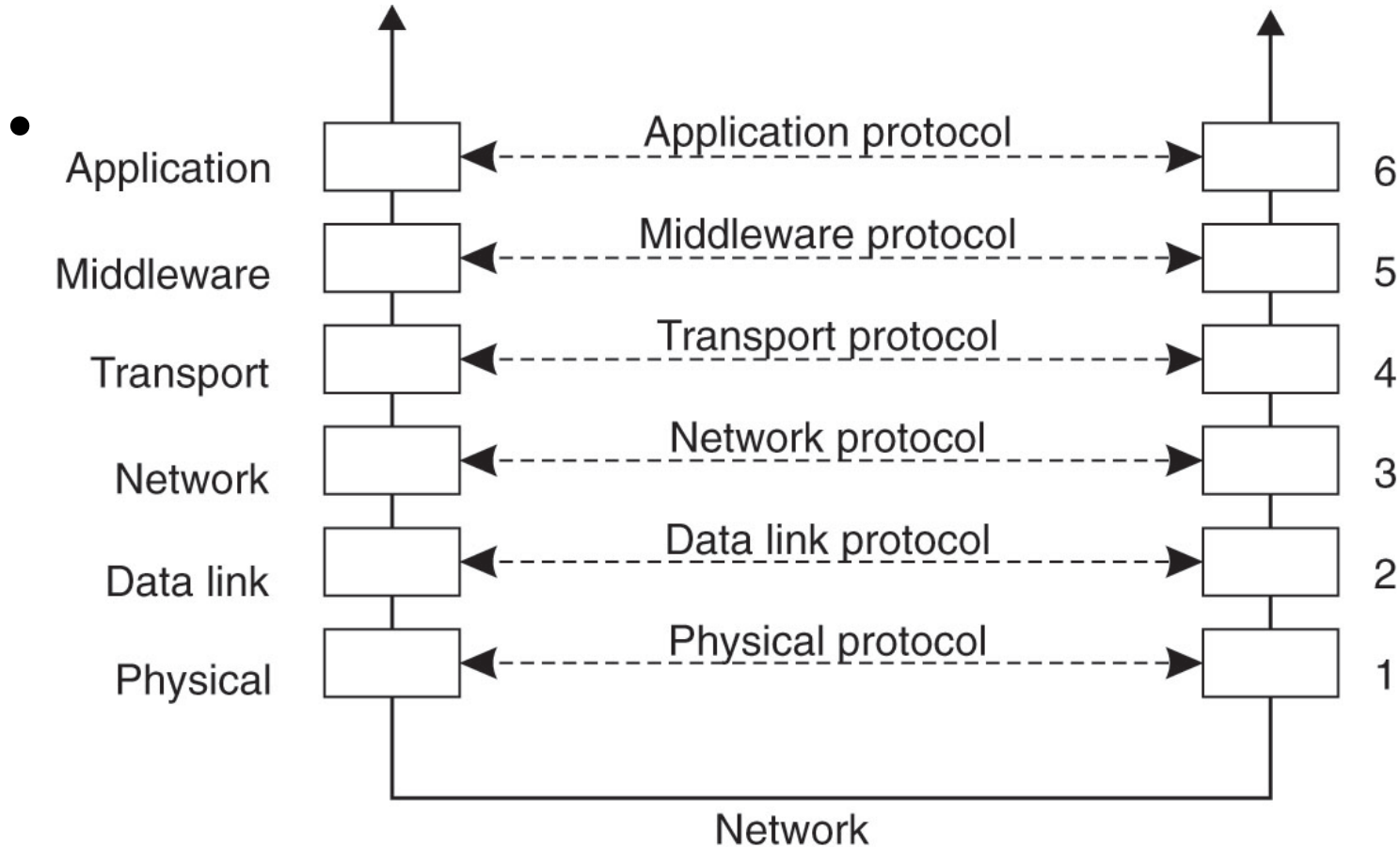
- Layers, interfaces, and protocols in the OSI model.

Layered Protocols (2)

- A typical message as it appears on the network.

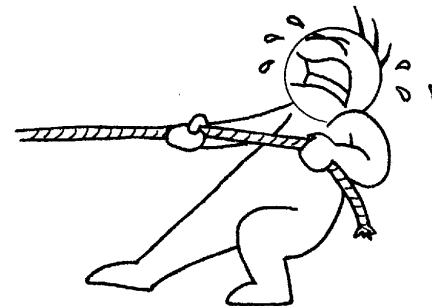
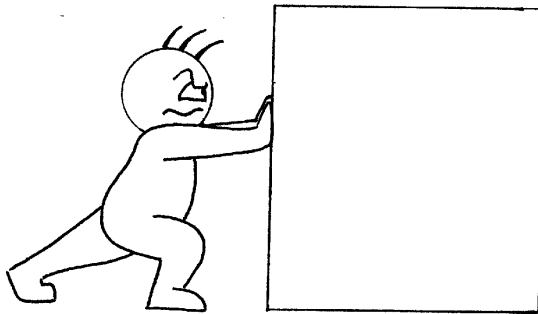


Middleware Protocols

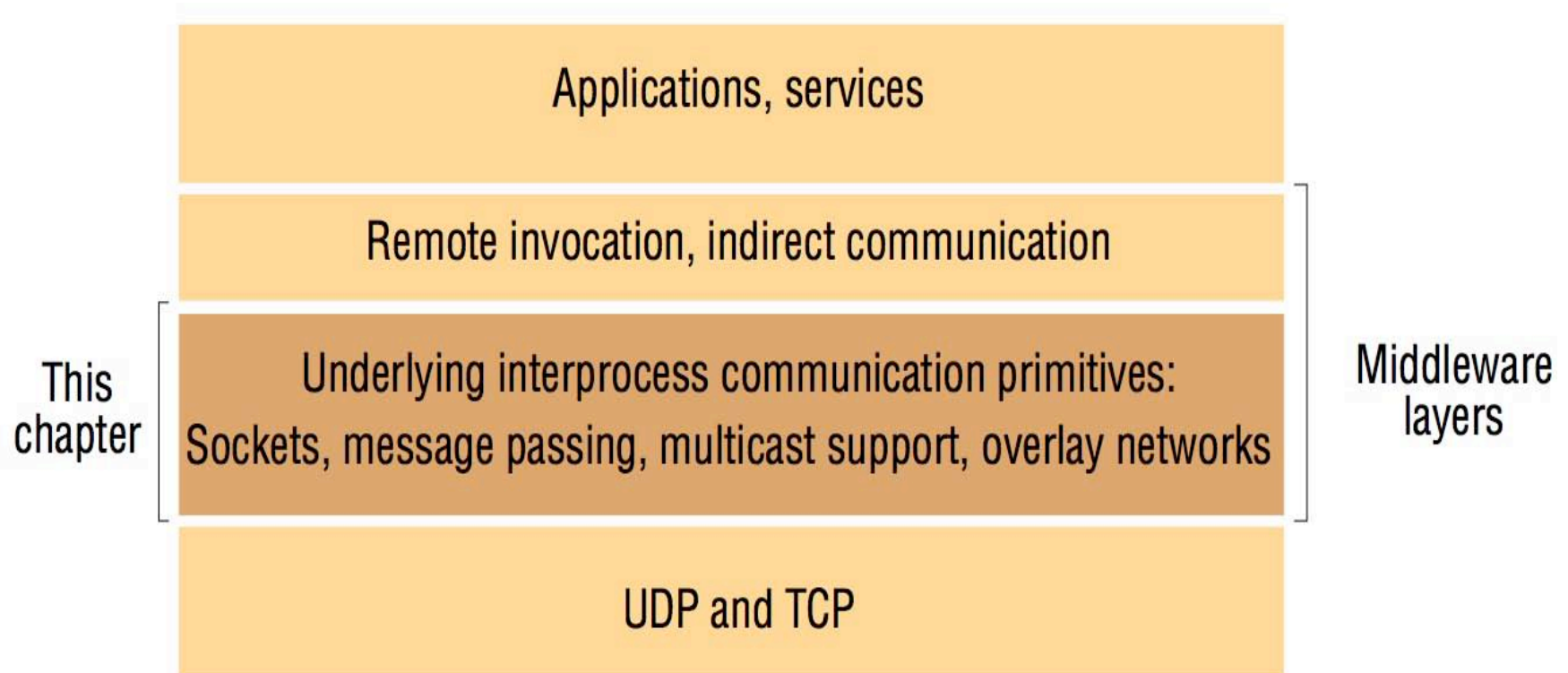


Types of Communication

- Synchronous / asynchronous
- Pull / Push
- Transient / Persistent
- Stateless / Stateful



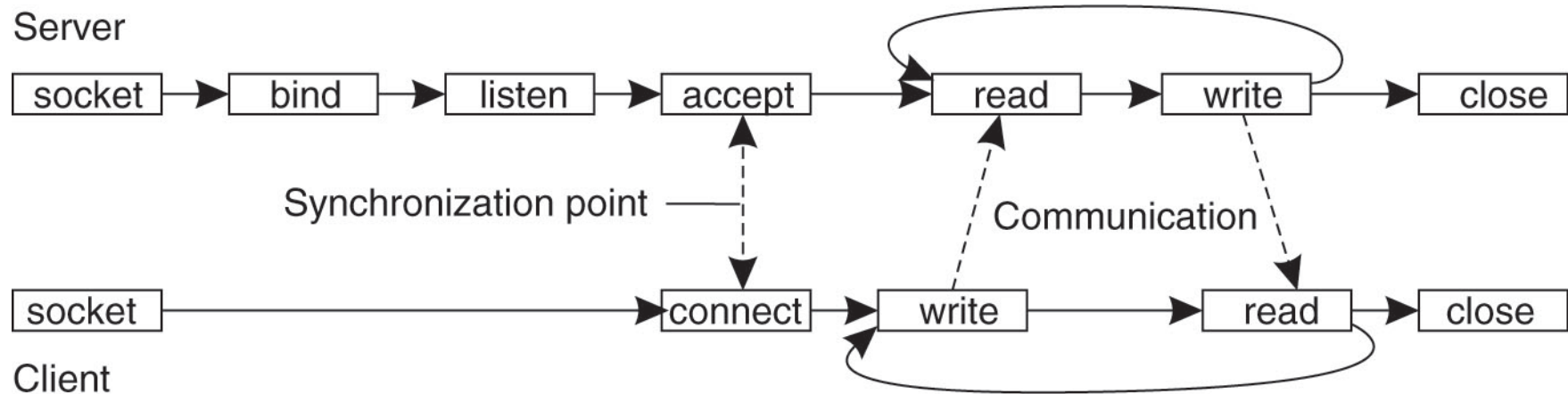
Types of Communication



Berkeley Sockets

Primitive	Meaning
Socket	Create a new communication end point
Bind	Attach a local address to a socket
Listen	Announce willingness to accept connections
Accept	Block caller until a connection request arrives
Connect	Actively attempt to establish a connection
Send	Send some data over the connection
Receive	Receive some data over the connection
Close	Release the connection

The Message-Passing Interface (1)



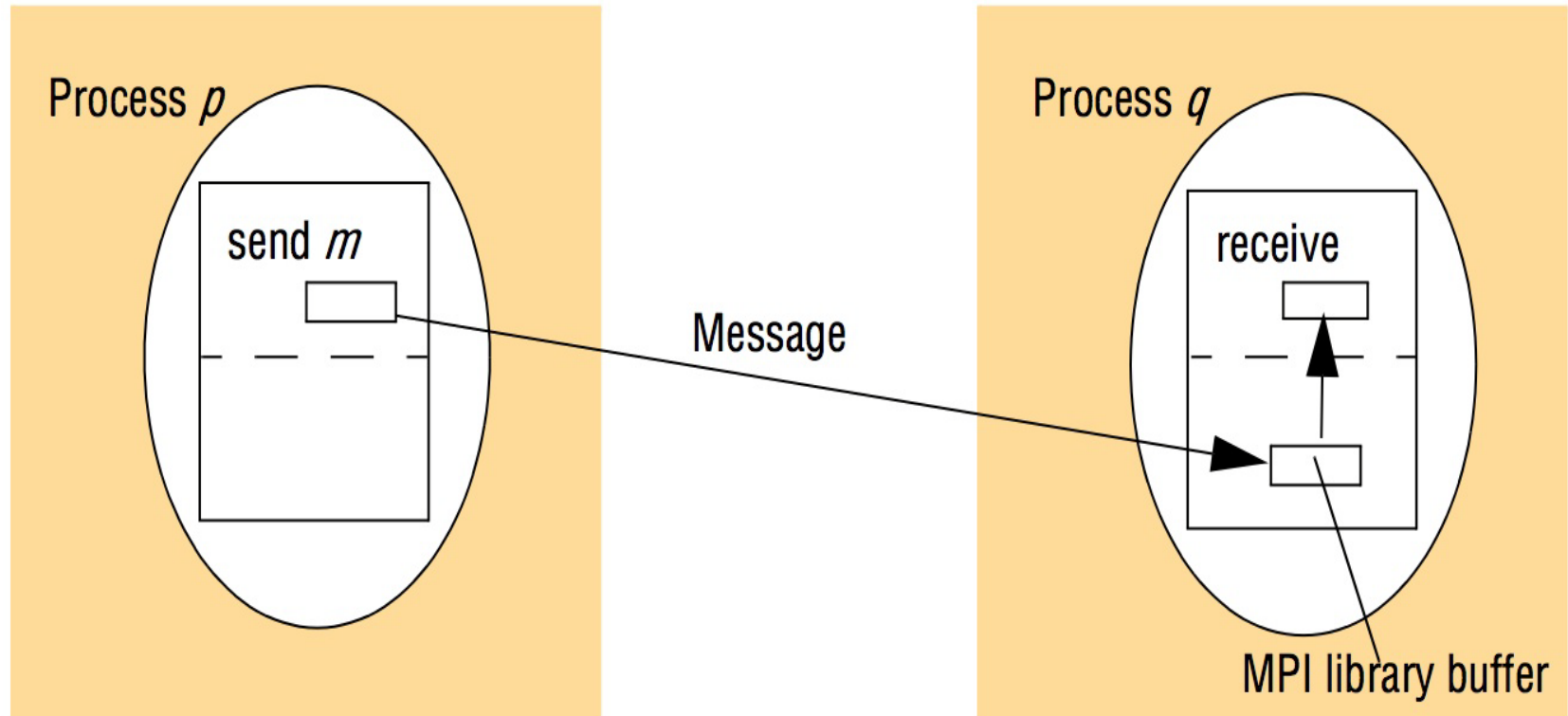
- Connection-oriented communication pattern using sockets.

The Message-Passing Interface (2)

Primitive	Meaning
MPI_bsend	Append outgoing message to a local send buffer
MPI_send	Send a message and wait until copied to local or remote buffer
MPI_ssend	Send a message and wait until receipt starts
MPI_sendrecv	Send a message and wait for reply
MPI_issend	Pass reference to outgoing message, and continue
MPI_issend	Pass reference to outgoing message, and wait until receipt starts
MPI_recv	Receive a message; block if there is none
MPI_irecv	Check if there is an incoming message, but do not block

- Some of the most intuitive message-passing primitives of MPI.

The Message-Passing Interface (3)

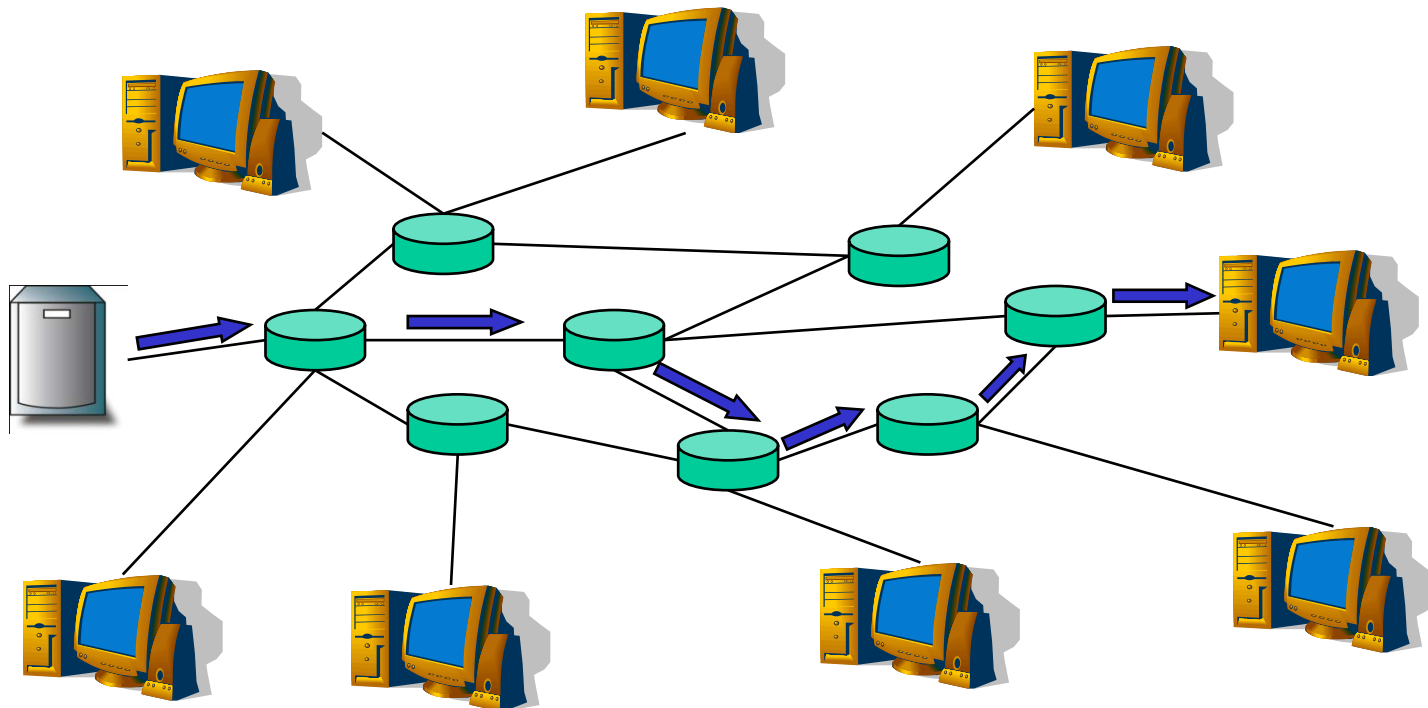


The Message-Passing Interface (4)

<i>Send operations</i>	<i>Blocking</i>	<i>Non-blocking</i>
<i>Generic</i>	<i>MPI_Send</i> : the sender blocks until it is safe to return – that is, until the message is in transit or delivered and the sender's application buffer can therefore be reused.	<i>MPI_Isend</i> : the call returns immediately and the programmer is given a communication request handle, which can then be used to check the progress of the call via <i>MPI_Wait</i> or <i>MPI_Test</i> .
<i>Synchronous</i>	<i>MPI_Ssend</i> : the sender and receiver synchronize and the call only returns when the message has been delivered at the receiving end.	<i>MPI_Issend</i> : as with <i>MPI_Isend</i> , but with <i>MPI_Wait</i> and <i>MPI_Test</i> indicating whether the message has been delivered at the receive end.
<i>Buffered</i>	<i>MPI_Bsend</i> : the sender explicitly allocates an MPI buffer library (using a separate <i>MPI_Buffer_attach</i> call) and the call returns when the data is successfully copied into this buffer.	<i>MPI_Ibsend</i> : as with <i>MPI_Isend</i> but with <i>MPI_Wait</i> and <i>MPI_Test</i> indicating whether the message has been copied into the sender's MPI buffer and hence is in transit.
<i>Ready</i>	<i>MPI_Rsend</i> : the call returns when the sender's application buffer can be reused (as with <i>MPI_Send</i>), but the programmer is also indicating to the library that the receiver is ready to receive the message, resulting in potential optimization of the underlying implementation.	<i>MPI_Irsend</i> : the effect is as with <i>MPI_Isend</i> , but as with <i>MPI_Rsend</i> , the programmer is indicating to the underlying implementation that the receiver is guaranteed to be ready to receive (resulting in the same optimizations),

Multicasting

- Communication can be of three types: unicast, broadcast and multicast.
 - Unicast:
 - One to one communication between hosts.
 - The destination address is the address of the unique host which is the intended recipient of the message.



Introduction to Multicasting

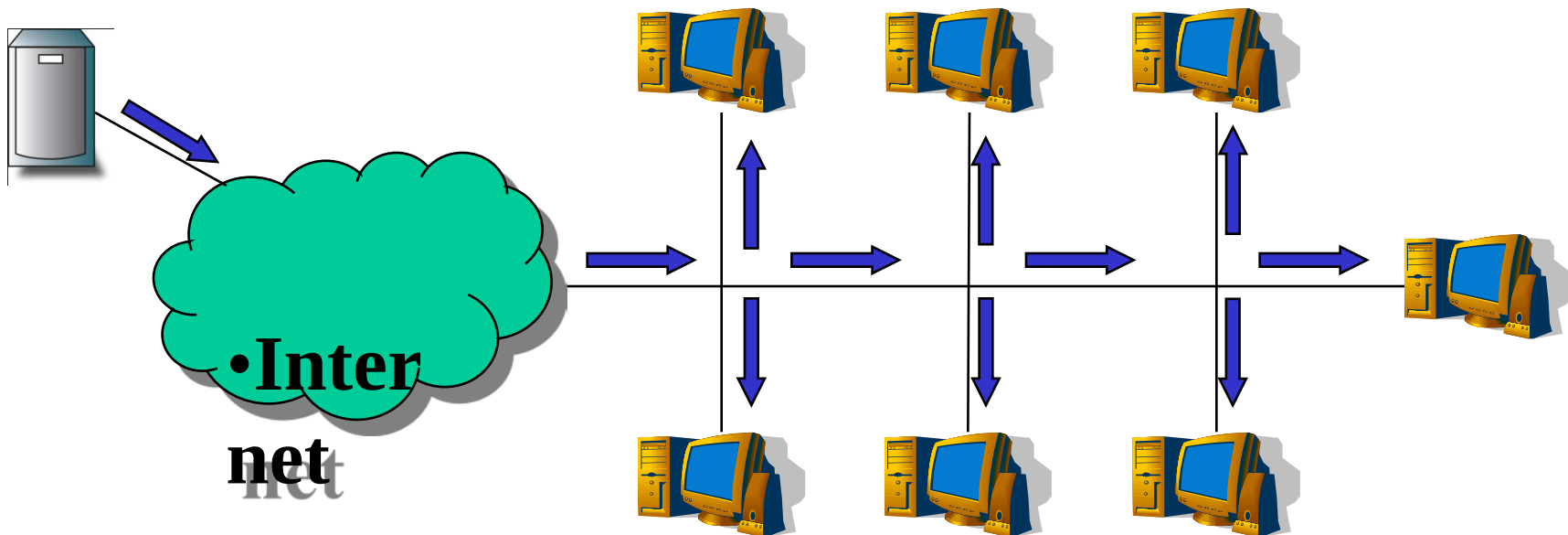
– Broadcast:

- One to all communication between hosts on a given network.
- The destination address is the address of the network.

•**Network**

•11111111

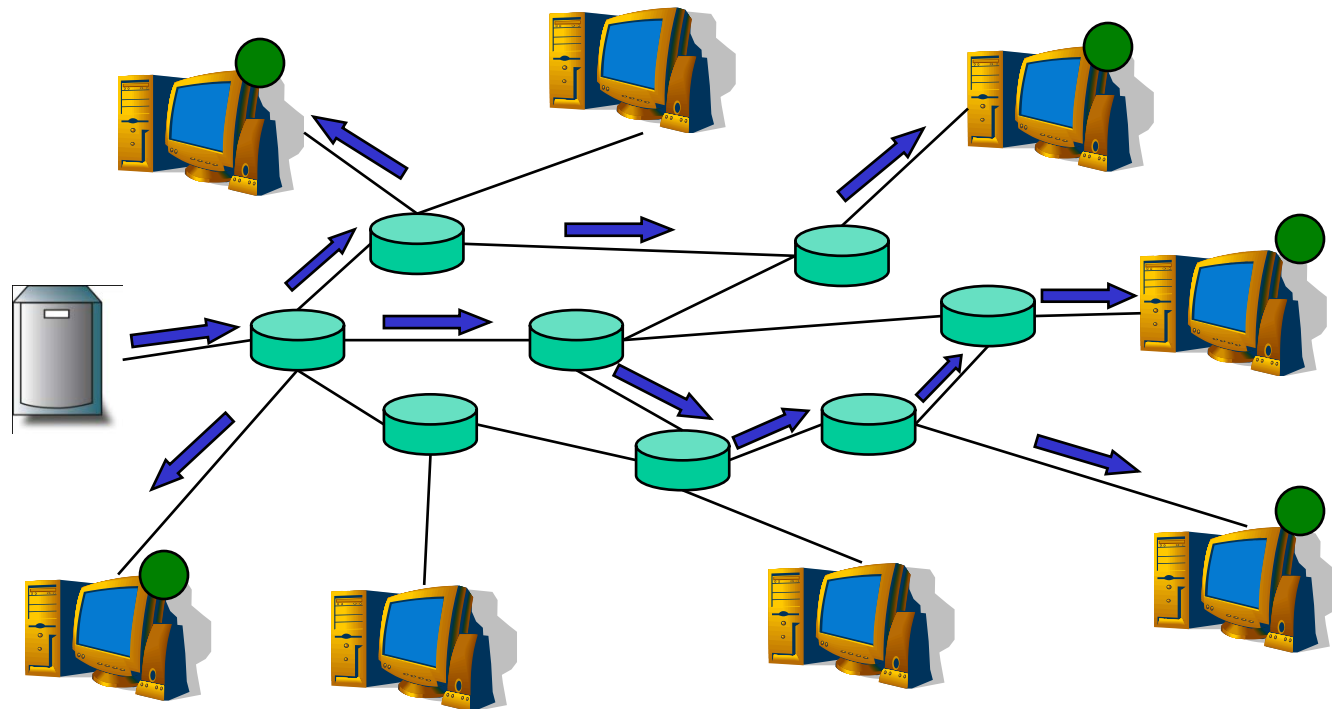
- Broadcast to all hosts on the Internet is not possible.



Introduction to Multicasting

– Multicast:

- One to many (but not all) communication between hosts.
- A single packet is transmitted to a group of recipients.
- The destination address is the address of the group.



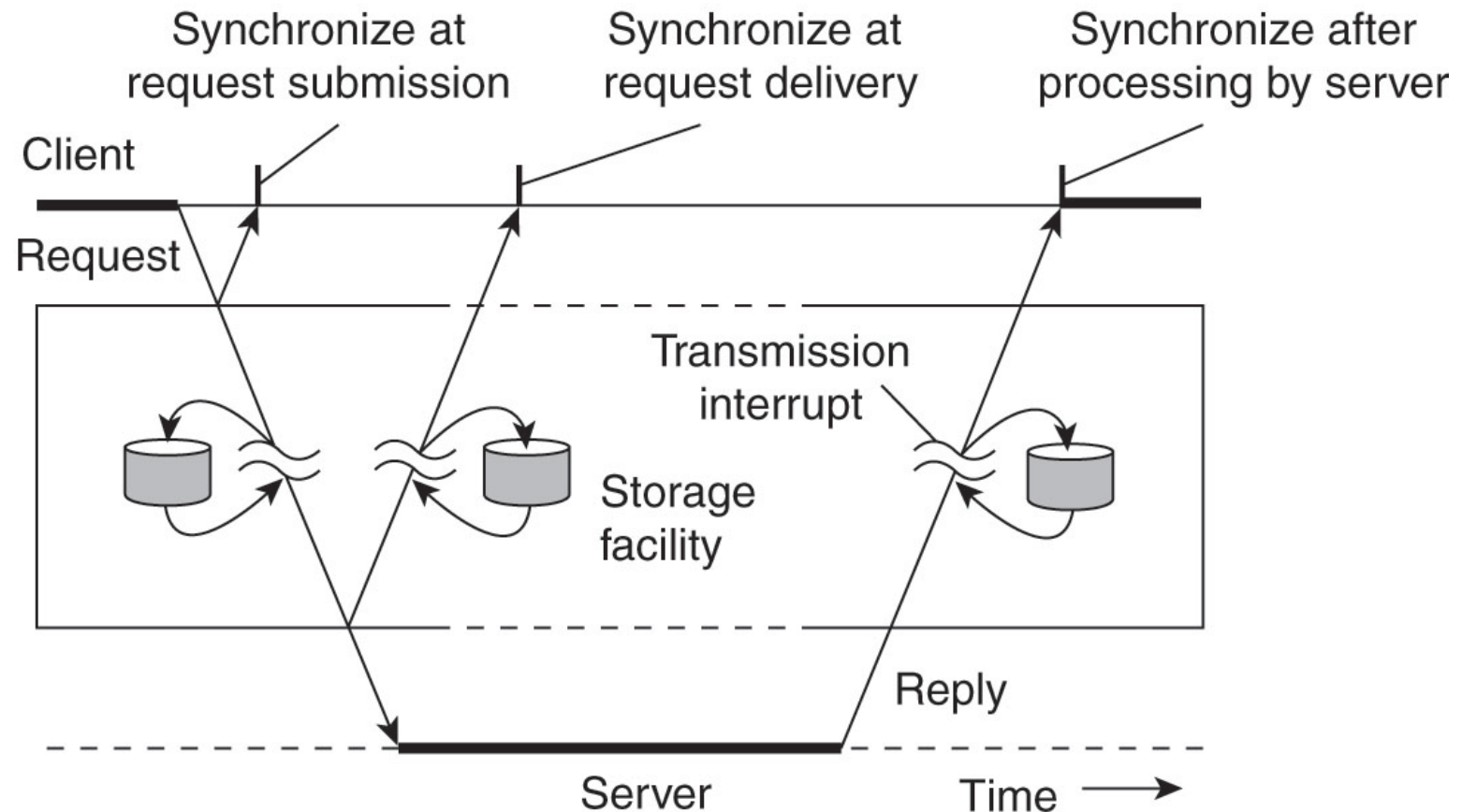
● • Hosts which are part of the group

Multicast vs. Multiple Unicast

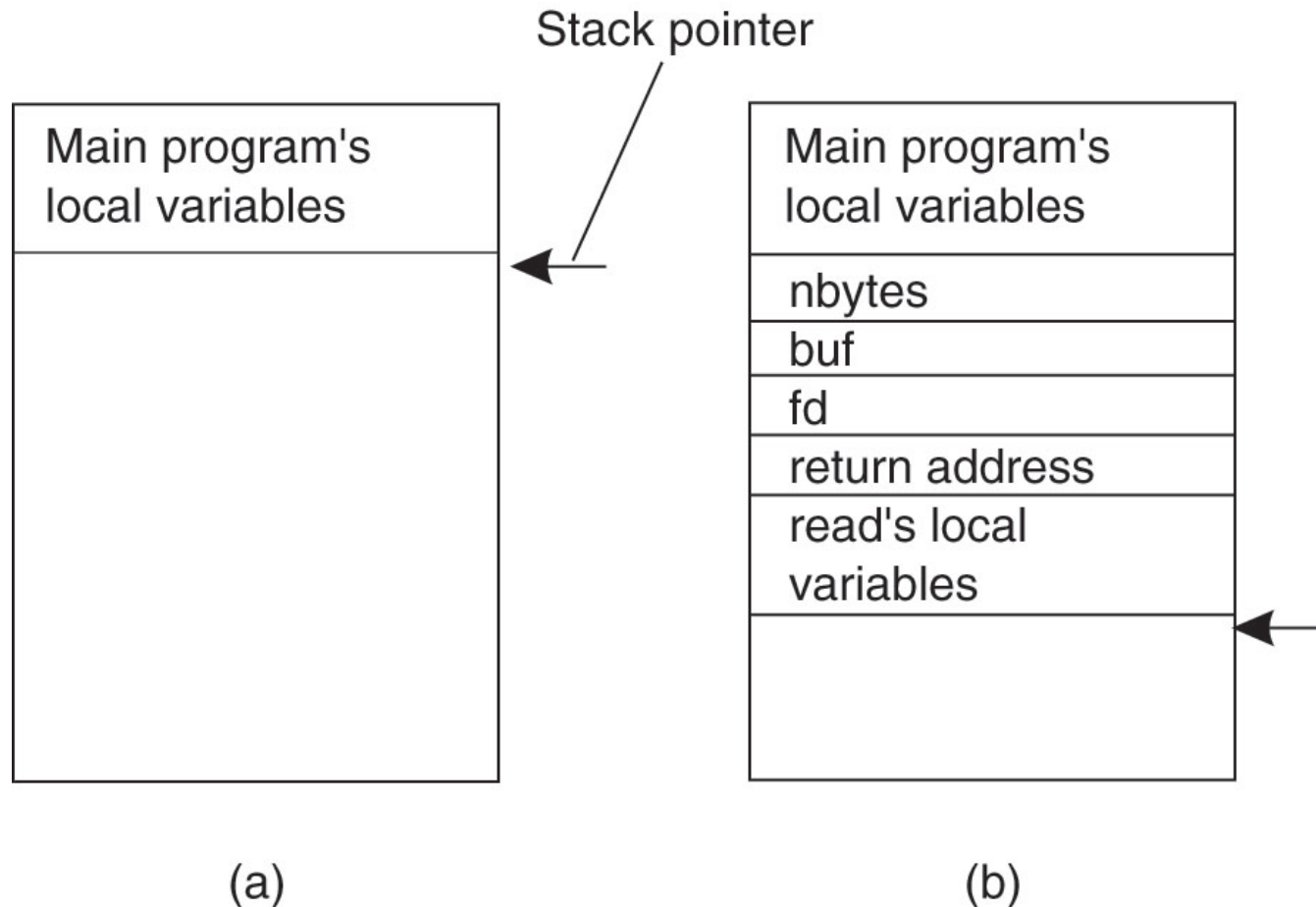
Multicasting:	Multiple Unicasting:
■ The source transmits a single packet which is duplicated by the routers.	■ The source transmits multiple packets.
■ Destination address is the same for all the duplicates.	■ Each packet is intended for a particular destination. ■ Each packet has a different unicast destination address.
■ Only one copy of a packet travels between any two routers.	■ Multiple copies may travel between any two routers.

Types of Communication

- Viewing middleware as an intermediate (distributed) service in application-level communication.

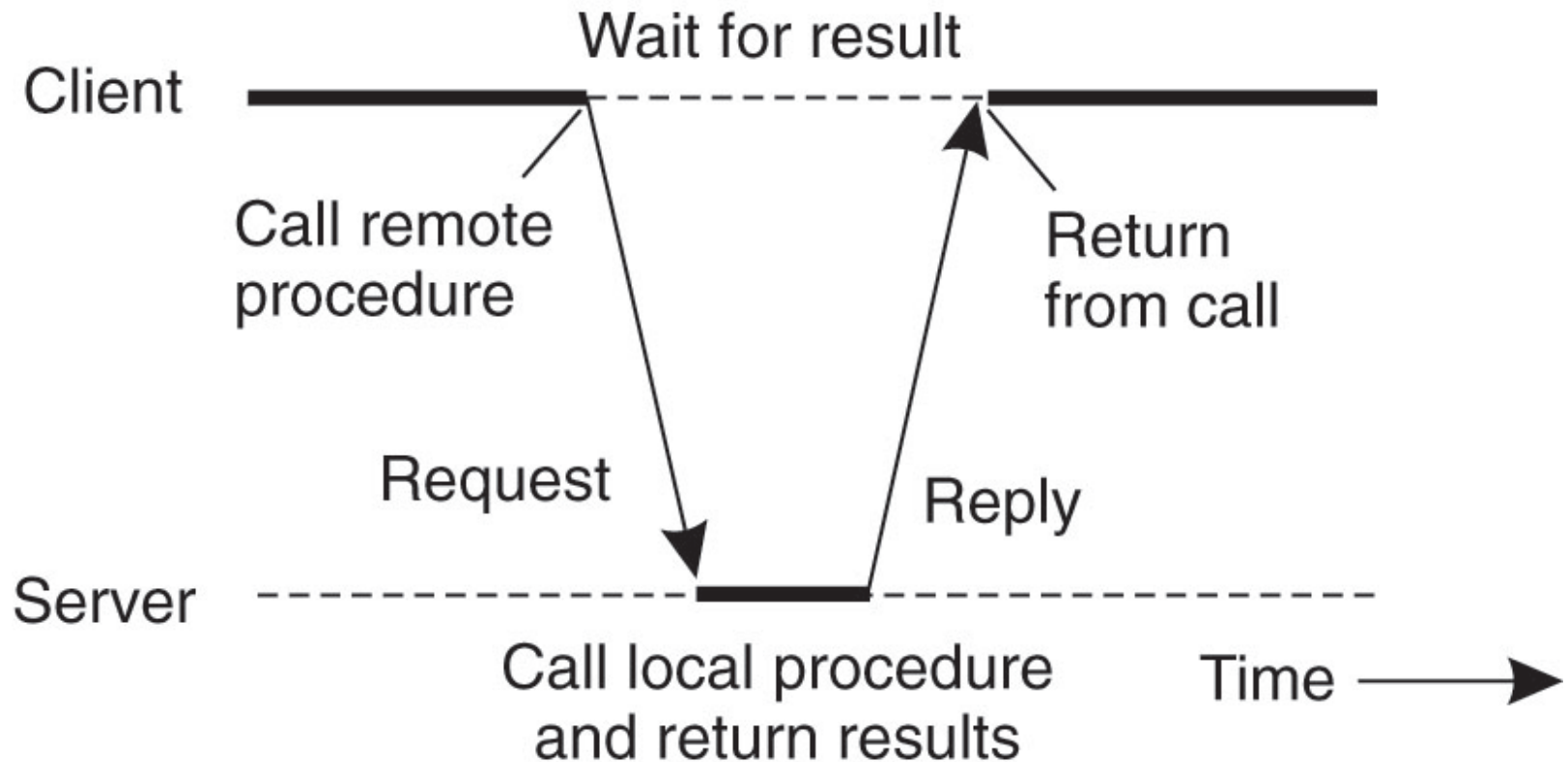


Conventional Procedure Call



- (a) Parameter passing in a local procedure call: the stack before the call to read. (b) The stack while the called procedure is active.

Client and Server Stubs



- Principle of RPC between a client and server program.

Remote Procedure Calls (1)

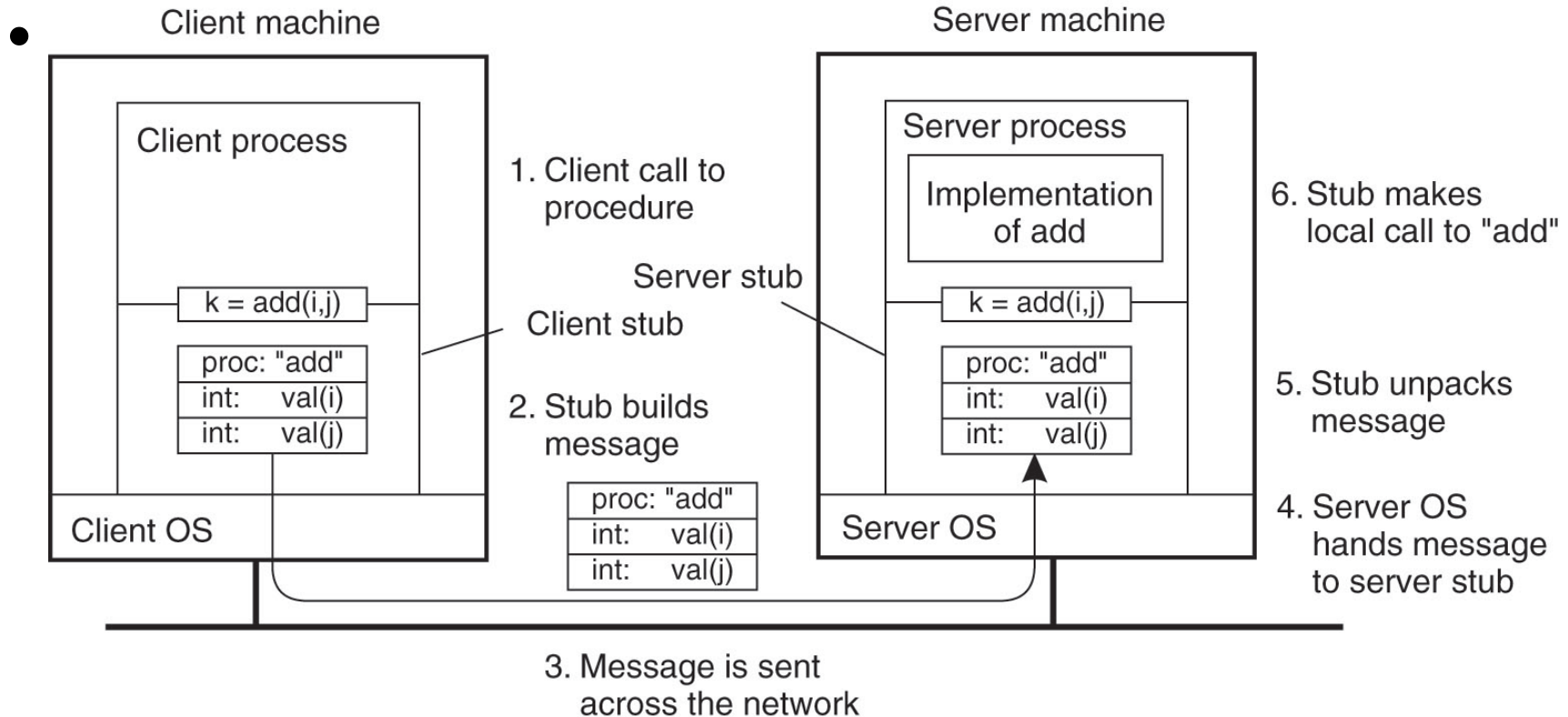
- **A remote procedure call occurs in the following steps:**
 1. The client procedure calls the client stub in the normal way.
 2. The client stub builds a message and calls the local operating system.
 3. The client's OS sends the message to the remote OS.
 4. The remote OS gives the message to the server stub.
 5. The server stub unpacks the parameters and calls the server.

Continued ...

Remote Procedure Calls (2)

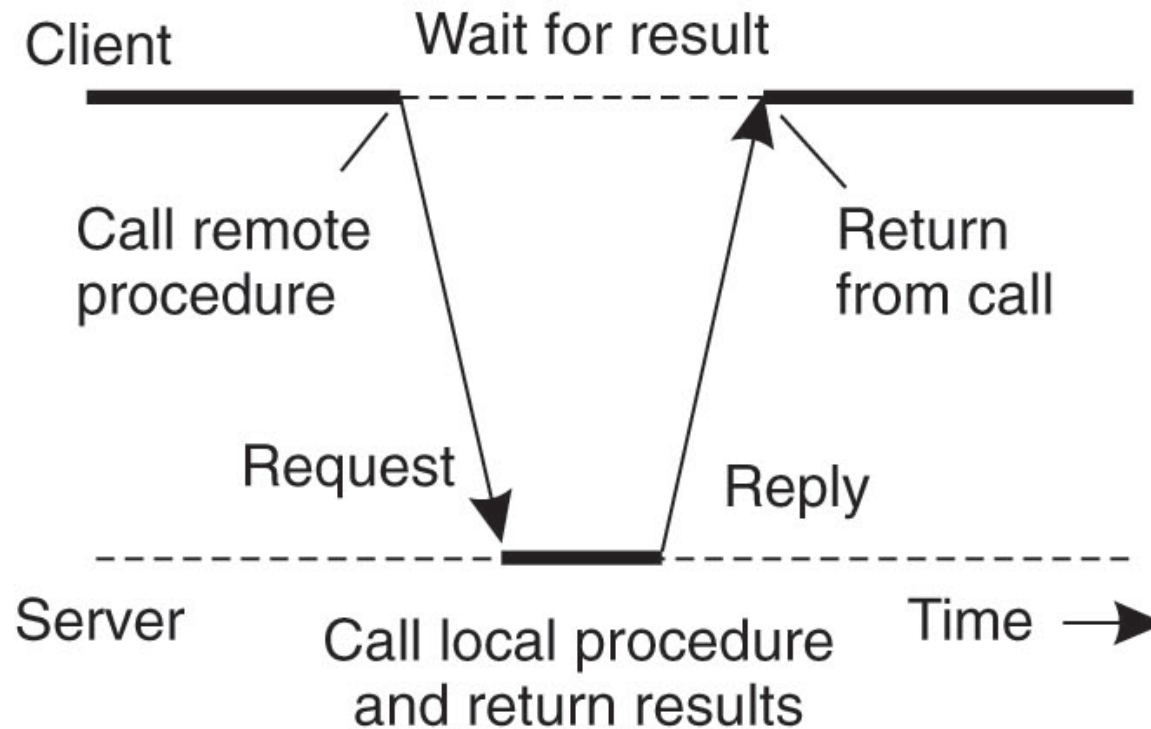
- **A remote procedure call occurs in the following steps (continued):**
 6. The server does the work and returns the result to the stub.
 7. The server stub packs it in a message and calls its local OS.
 8. The server's OS sends the message to the client's OS.
 9. The client's OS gives the message to the client stub.
 10. The stub unpacks the result and returns to the client.

Passing Value Parameters (1)



Asynchronous RPC (1)

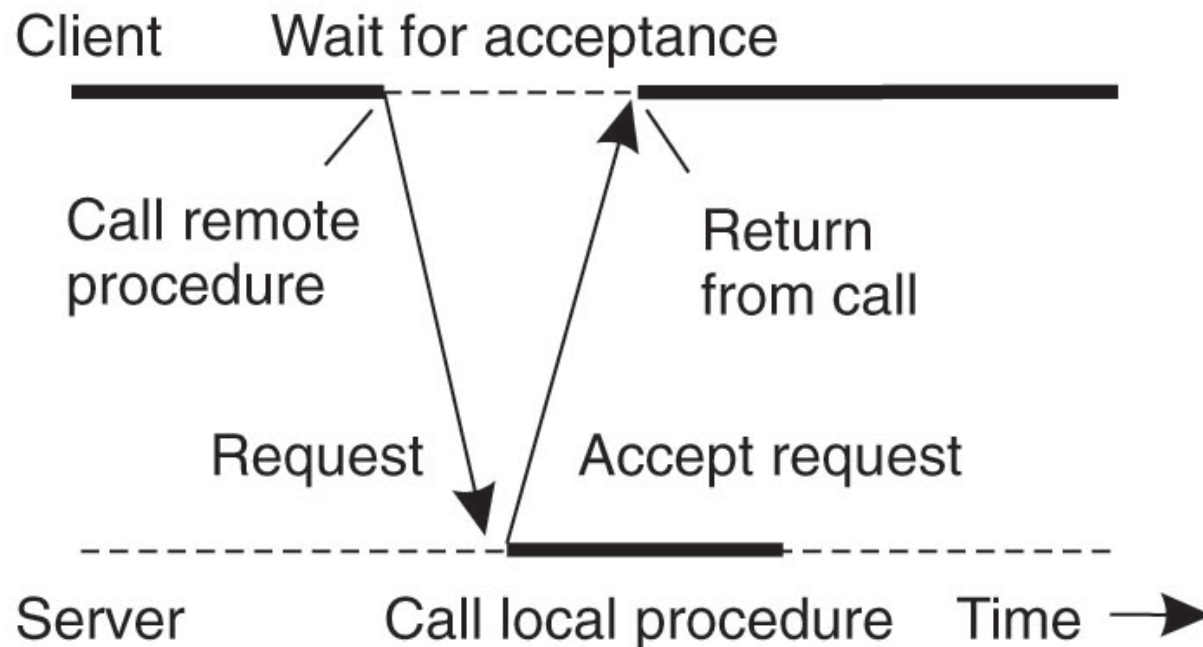
- (a) The interaction between client and server in a traditional RPC.



(a)

Asynchronous RPC (2)

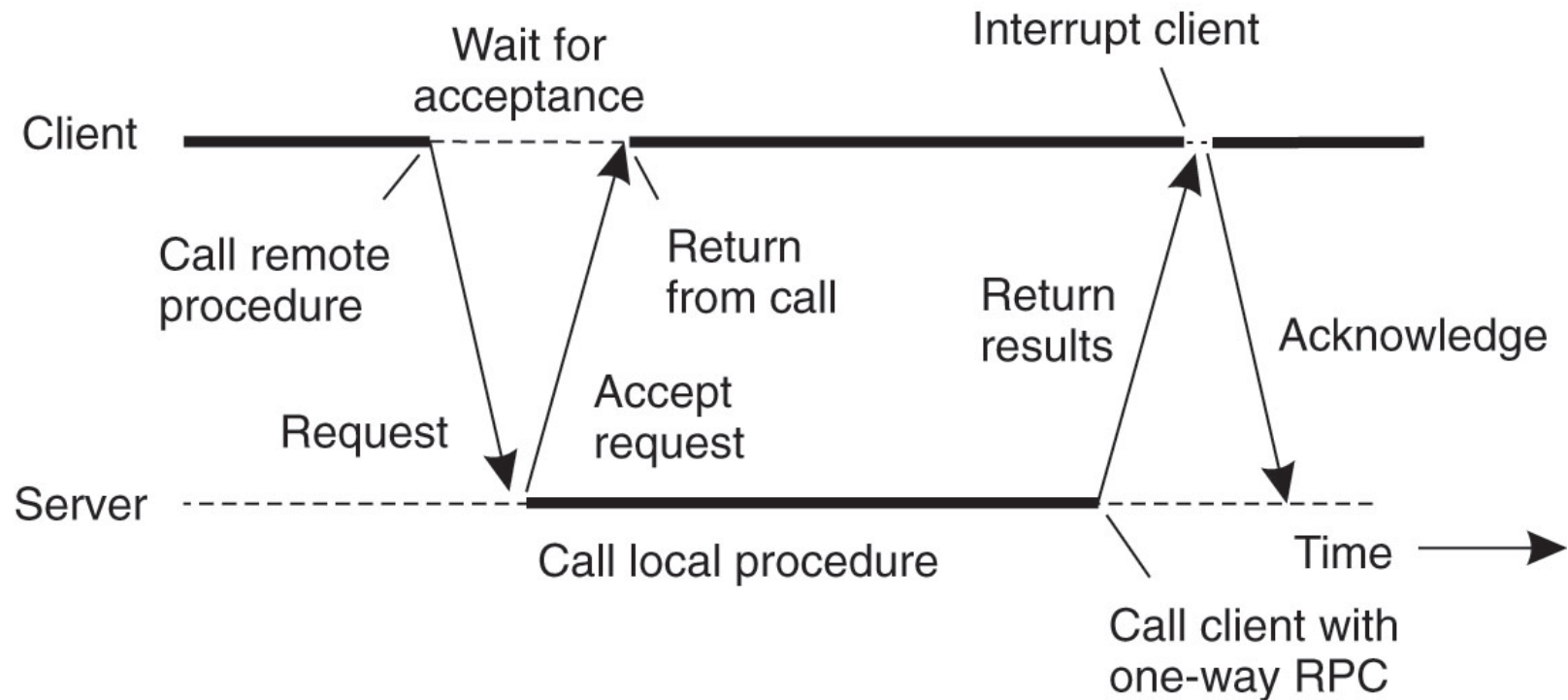
- (b) The interaction using asynchronous RPC.



(b)

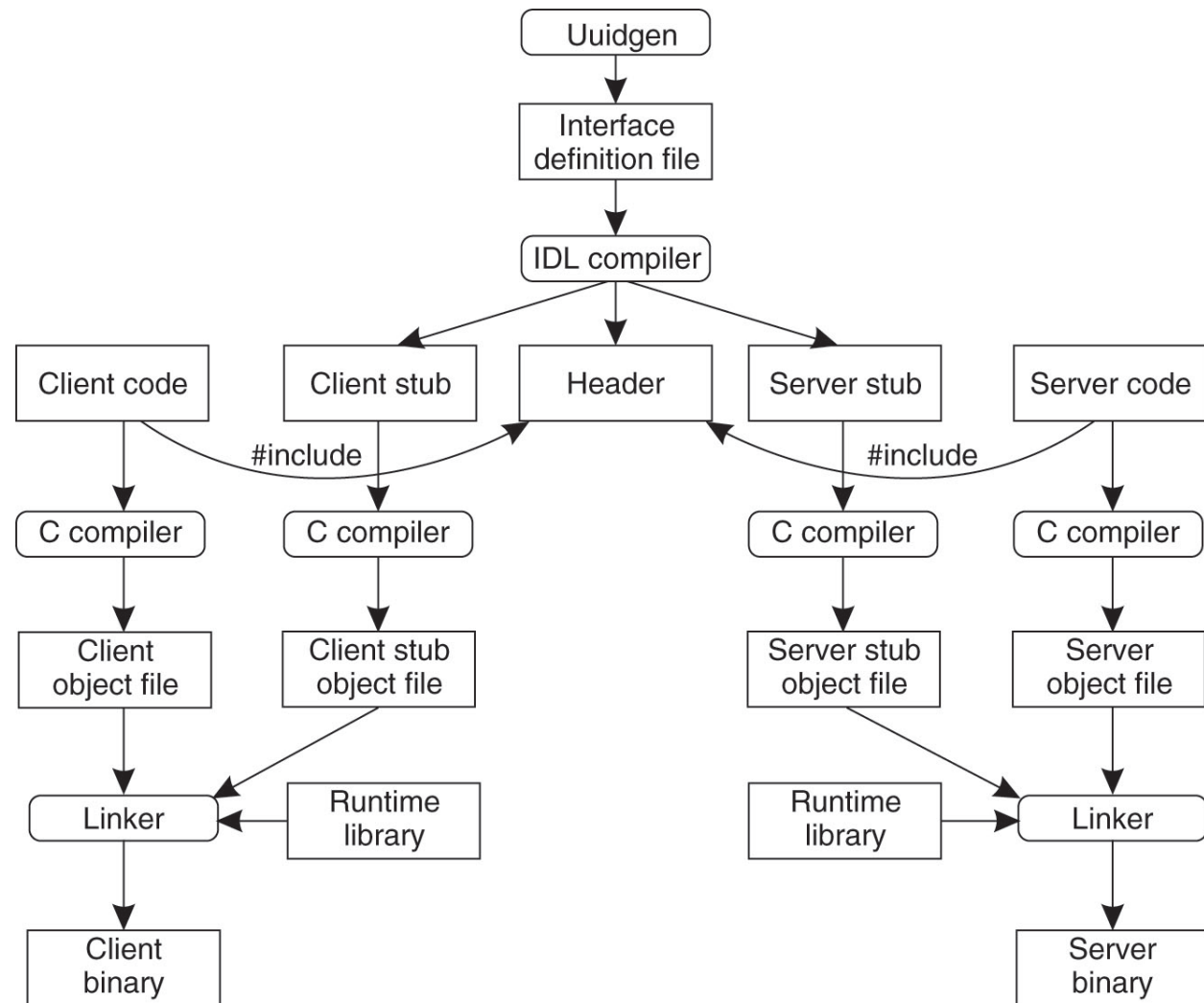
Asynchronous RPC (3)

- A client and server interacting through two asynchronous RPCs.



Writing a Client and a Server (1)

- The steps in writing a client and a server in DCE RPC.



Writing a Client and a Server (2)

- Three files output by the IDL compiler:
- A header file (e.g., interface.h, in C terms).
- The client stub.
- The server stub.

Binding a Client to a Server (1)

- Registration of a server makes it possible for a client to locate the server and bind to it.
- Server location is done in two steps:
 1. Locate the server's machine.
 2. Locate the server on that machine.

Binding a Client to a Server (2)

- Client-to-server binding in DCE.

